# CSC 370: Databases Constraints in Relational Algebra

Sean Chester • University of Victoria

# Which relations below are BCNF?

$AB \rightarrow C$

$BC \rightarrow D$

$D \rightarrow C$

$C \rightarrow A$

- R1(A, B, C)
- R2(B, C, D)
- R3(A)
- R4(A, B, C, D)
- R5(E, F)
- R6(A, C, D)

## Today's Goal:

**You can document constraints in relational algebra, including key and referential integrity constraints**

## Outline

1. Review BCNF decomposition
2. Why do we need foreign keys?
3. Relational algebra as a constraint language
4. 7 worked examples

# Last class...

# A *key* of a relation

Consider a relation R(A,B,C,D) with the set of FD's that we have been using:
$A \rightarrow C$
$BC \rightarrow D$
$C \rightarrow B$

He have already computed that:

- $\{A\}^+ = \{A,B,C,D\}$
- $\{B,C\}^+ = \{B,C,D\}$
- $\{A, C\}^+ = \{A,B,C,D\}$

Therefore, we can see that:

- {A} is a *key* and a *superkey* for R
- {A,C} is a *superkey* for R but not a key because it is a superset of superkey {A}, i.e., it is not minimal
- {B,C} is neither a key nor a superkey for R, because it cannot determine A

Source: Garcia-Molina et al. (2007), §3.1.2

# Checking that our example is in BCNF

We have one functional dependency: v_number → name

Before decomposition, we see that there are attributes (namely *class*) that are not determined by the antecedent (namely *v_number*) of this FD

After decomposition, the first relation no longer contains *class*, so v_number is a (super)key. The FD does not apply to the second relation, because the second relation does not have the attribute *name*

**Before**

| v_number | name | class |
|---|---|---|
| V00111222 | Alice | CSC 370 |
| V00111222 | Alice | CSC 320 |
| V00333444 | Bob | CSC 370 |

**After**

| v_number | name |
|---|---|
| V00111222 | Alice |
| V00333444 | Bob |

| v_number | class |
|---|---|
| V00111222 | CSC 370 |
| V00111222 | CSC 320 |
| V00333444 | CSC 370 |

Source: Garcia-Molina et al. (2007), §3.3.3

# What is “Decomposition”?

Consider that you have a relation **R** with attributes $\mathbf{A=\{A_1, A_2, ..., A_n\}}$

A *decomposition* of **R** splits it into two relations:

**S** with attributes $\mathbf{B=\{B_1, B_2, ..., B_m\}}$ and
**T** with attributes $\mathbf{C=\{C_1, C_2, ..., C_k\}}$.

Such that:

- $\mathbf{A = B \cup C}$
- $\mathbf{S = \pi_{(B1, B2, ..., Bm)}(R)}$
- $\mathbf{T = \pi_{(C1, C2, ..., Cm)}(R)}$

Why would we do that?

Source: Garcia-Molina et al. (2007), §3.3.2

# BCNF Decomposition Algorithm

What is our systematic process (i.e., algorithm) to arrive at a decomposition in BCNF?

**BCNF_Decomp($R_0$, $F_0$)**
Input: Relation $R_0$ with attribute set $C_0$ and functional dependencies $F_0$
Output: A decomposition of $R_0$ in which all relations are in BCNF

1. If $R_0$ is in BCNF, return $R_0$
2. Select a BCNF violation (an FD for which the antecedent is not a superkey), $X \rightarrow Y$
3. Compute the closure $X^+$
4. Let $R_1 := X^+$
5. Let $R_2 := ( C \setminus X^+ ) \cup X$
6. Project $F_0$ to get functional dependencies for $R_1$ and $R_2$ denoted $F_1$ and $F_2$, respectively
7. Return **BCNF_Decomp**($R_1$, $F_1$) $\cup$ **BCNF_Decomp**($R_2$, $F_2$)

Source: Garcia-Molina et al. (2007), §3.3.4

# Small Decomposition Example 1

Let's decompose this together:

| Schema: | Functional Dependencies |
|---|---|
| **R(A, B, C)** | {A}→{B} |
| | {B}→{C} |

How do we start?

1. Check if {A} and {B} are superkeys, i.e., does $\{A\}^+=\{A,B,C\}$ and $\{B\}^+=\{A,B,C\}$

   **NO!** A cannot be determined from B. Thus the FD {B}→{C} is a *BCNF violation*
2. Calculate sets $R_1 = \{B\}^+$ and $R_2 = \{A,B,C\} \setminus \{B\}^+ \cup \{B\}$
3. Project the FD's onto each relation to get $F_1 = \{\{A\}\rightarrow\{B\}\}$ and $F_2 = \{\{B\}\rightarrow\{C\}\}$

**Final BCNF Decomposition**: $R_1(A,B)$ and $R_2(B,C)$

Source: Garcia-Molina et al. (2007), §3.3.4

# Lossless Decompositions

- What if we want our original data back?
  - Members ⋈ Accounts

| member_id | sin | date_joined |
|---|---|---|
| 000111 | 700100200 | 2011-01-08 |
| 000222 | 912345678 | 2020-03-21 |
| 000333 | 700300400 | 2021-09-20 |
| 000444 | 700500600 | 2000-04-02 |

| member_id | acc_id | bal | opened |
|---|---|---|---|
| 000111 | 10023 | $5200 | 2011-01-08 |
| 000111 | 10187 | $0 | 2011-01-08 |
| 000222 | 10331 | $100 | 2020-03-23 |
| 000333 | 10855 | $37910 | 2021-09-20 |
| 000444 | 10001 | $12500 | 2000-04-02 |
| 000444 | 10002 | $2500 | 2000-04-02 |

```
SELECT *
FROM Members
NATURAL JOIN Accounts;
```

Source: Garcia-Molina et al. (2007), §3.3.4

# Why do we need foreign keys?

# Do we have enough constraints?

Members(member_id, sin, date_joined)
Accounts(account_id, member_id, bal, opened)

| member_id | sin | date_joined |
|---|---|---|
| 000111 | 700100200 | 2011-01-08 |
| 000222 | 912345678 | 2020-03-21 |
| 000333 | 700300400 | 2021-09-20 |
| 000444 | 700500600 | 2000-04-02 |

| member_id | acc_id | bal | opened |
|---|---|---|---|
| 000111 | 10023 | $5200 | 2011-01-08 |
| 000111 | 10187 | $0 | 2011-01-08 |
| 000222 | 10331 | $100 | 2020-03-23 |
| 000333 | 10855 | $37910 | 2021-09-20 |
| 000444 | 10001 | $12500 | 2000-04-02 |
| 000444 | 10002 | $2500 | 2000-04-02 |

# Do we have enough constraints?

Members(member_id, sin, date_joined)
Accounts(account_id, member_id, bal, opened)

| member_id | sin | date_joined |
|---|---|---|
| 000111 | 700100200 | 2011-01-08 |
| 000222 | 912345678 | 2020-03-21 |
| 000333 | 700300400 | 2021-09-20 |
| 000444 | 700500600 | 2000-04-02 |

- How do we know those are the same member ids? Cause the attribute names are the same?

| member_id | acc_id | bal | opened |
|---|---|---|---|
| 000111 | 10023 | $5200 | 2011-01-08 |
| 000111 | 10187 | $0 | 2011-01-08 |
| 000222 | 10331 | $100 | 2020-03-23 |
| 000333 | 10855 | $37910 | 2021-09-20 |
| 000444 | 10001 | $12500 | 2000-04-02 |
| 000444 | 10002 | $2500 | 2000-04-02 |

# Do we have enough constraints?

Members(member_id, sin, date_joined)
Accounts(account_id, member_id, bal, opened)

| member_id | sin | date_joined |
|---|---|---|
| 000111 | 700100200 | 2011-01-08 |
| 000222 | 912345678 | 2020-03-21 |
| 000333 | 700300400 | 2021-09-20 |
| 000444 | 700500600 | 2000-04-02 |

| member_id | acc_id | bal | opened |
|---|---|---|---|
| 000111 | 10023 | $5200 | 2011-01-08 |
| 000111 | 10187 | $0 | 2011-01-08 |
| 000222 | 10331 | $100 | 2020-03-23 |
| 000333 | 10855 | $37910 | 2021-09-20 |
| 000444 | 10001 | $12500 | 2000-04-02 |
| 000444 | 10002 | $2500 | 2000-04-02 |

- Is this valid?

```
INSERT INTO `Accounts`
VALUES ('000555', 10003, 0, 1999-12-31);
```

# Do we have enough constraints? Ref. Integrity!

Members(member_id, sin, date_joined)
Accounts(account_id, member_id, bal, opened)

| member_id | sin | date_joined |
|---|---|---|
| 000111 | 700100200 | 2011-01-08 |
| 000222 | 912345678 | 2020-03-21 |
| 000333 | 700300400 | 2021-09-20 |
| 000444 | 700500600 | 2000-04-02 |

| member_id | acc_id | bal | opened |
|---|---|---|---|
| 000111 | 10023 | $5200 | 2011-01-08 |
| 000111 | 10187 | $0 | 2011-01-08 |
| 000222 | 10331 | $100 | 2020-03-23 |
| 000333 | 10855 | $37910 | 2021-09-20 |
| 000444 | 10001 | $12500 | 2000-04-02 |
| 000444 | 10002 | $2500 | 2000-04-02 |

- Is this valid? **Who is this??**

```
INSERT INTO `Accounts`
VALUES ('000555', 10003, 0, 1999-12-31);
```

# Why do we need referential integrity?

- ❏ To constrain what values are allowed in certain tables
- ❏ To document the way in which tables are related to each other
- ❏ To prevent deleting information that is needed in another table
- ❏ All of the above

# Relational algebra as a constraint language

# What is a *constraint*?

**Constraint**: "*A limitation on what data can be*" (pg. 18)

Ensuring that the constraints of a database are maintained is our best defense against the deterioration of data quality.

**Examples**:

- In a database of product sales, every *is_sold* value must be a boolean
- In a database of NHL hockey matches, no team can play against itself
- In an employee database, nobody's age can be > 85 or < 16
- In a library database, every ISBN must be unique
- In a university enrollment database, no student can be enrolled in *Databases* if they have not obtained at least a C grade in *Algorithms and Data Structures*
- In a pre-Y2K database, no year field can require more than two numeric digits
- In an undisclosed American flight booking database that hates international travellers, the phone field cannot be NULL and must be ten digits in the range [0,9] and cannot be 555-555-5555 or 123-456-7890.

Source: Garcia-Molina et al. (2007), §2.1.1

# How do we express a constraint in R.A.?

1. We can state that the result of a specific relational algebra expression is always equal to the empty set

For example:

*All workers in an employee database are younger than 85*

Is logically equivalent to:

*Selecting all employees from the database w/ age ≥ than 85, **must** yield an empty result.*

$$\sigma_{age \geq 85}(\text{Employees}) = \emptyset$$

Source: Garcia-Molina et al. (2007), §2.5.1

# How do we express a constraint in R.A.?

2. We can state that the result of a specific relational algebra expression is a subset of the result of another relational algebra expression

For example:

*Every flight must depart from an airport in our table of airports*

Is logically equivalent to:

*Selecting all flight departure airports **must yield a subset** of our list of airport codes.*

$$\pi_{\text{departure_airport}}(\text{Flights}) \subseteq \pi_{\text{airport_codes}}(\text{Airports})$$

Source: Garcia-Molina et al. (2007), §2.5.1

# Example Schema: NHL Hockey Matches

In the following examples, we will write constraints on the following relational schema for a database storing information about NHL hockey matches:

Team(id, city, name, year_founded)
Matches(home_team, away_team, date)  // semantically, should refer to Team.id

Source: Instructor

# Example 1: Value Range

**Constraint**: In a database of NHL hockey matches, no team existed prior to November 26, 1917
**Relations**: Team(id, city, name, year_founded)

Source: Garcia-Molina et al. (2007), §2.5.4

# Example 1: Value Range

**Constraint**: In a database of NHL hockey matches, no team existed prior to November 26, 1917
**Relations**: Team(id, city, name, year_founded)

1. "No team existed prior to November 26, 1917" can be rewritten more explicitly as:
    a. "There does not exist a team in which year_founded is before November 26, 1917"

This logical structure will lead to our constraint form: Query = ∅

Source: Garcia-Molina et al. (2007), §2.5.4

# Example 1: Value Range

**Constraint**: In a database of NHL hockey matches, no team existed prior to November 26, 1917
**Relations**: Team(id, city, name, year_founded)

1. "No team existed prior to November 26, 1917" can be rewritten more explicitly as:
   a. "There does not exist a team in which year_founded is before November 26, 1917"

2. In predicate logic: $\nexists t \in T$, t.year_founded < '1917-11-26', where T is the set of Teams

A mathematically precise condition can be easily converted into a query

Source: Garcia-Molina et al. (2007), §2.5.4

# Example 1: Value Range

**Constraint**: In a database of NHL hockey matches, no team existed prior to November 26, 1917
**Relations**: Team(id, city, name, year_founded)

1. "No team existed prior to November 26, 1917" can be rewritten more explicitly as:
   a. "There does not exist a team in which year_founded is before November 26, 1917"

2. In predicate logic: $\nexists t \in T$, t.year_founded < '1917-11-26', where T is the set of Teams

3. Which as a constraint can be written in relational algebra as:

$$\sigma_{\text{year_founded} < \text{'1917-11-26'}}(\text{Teams}) = \emptyset$$

t.year_founded < '1917-11-26' $\nexists t \in T$

Source: Garcia-Molina et al. (2007), §2.5.4

# Example 2: Attribute Relationship

**Constraint**: In a database of NHL hockey matches, no team can play against itself
**Relations**: Matches(home_team, away_team, date)

Source: Garcia-Molina et al. (2007), §2.5.4

# Example 2: Attribute Relationship

**Constraint**: In a database of NHL hockey matches, no team can play against itself
**Relations**: Matches(home_team, away_team, date)

1. "No team can play against itself" can be rewritten more explicitly as:
   a. "There does not exist a match in which the home team and away team are the same"

This logical structure will lead to our constraint form: Query = ∅

Source: Garcia-Molina et al. (2007), §2.5.4

# Example 2: Attribute Relationship

**Constraint**: In a database of NHL hockey matches, no team can play against itself
**Relations**: Matches(home_team, away_team, date)

1. "No team can play against itself" can be rewritten more explicitly as:
   a. "There does not exist a match in which the home team and away team are the same"

2. In predicate logic: $\nexists m \in M$, m.home_team=m.away_team, where M is the set of Matches

A mathematically precise condition can be easily converted into a query

Source: Garcia-Molina et al. (2007), §2.5.4

# Example 2: Attribute Relationship

**Constraint**: In a database of NHL hockey matches, no team can play against itself
**Relations**: Matches(home_team, away_team, date)

1. "No team can play against itself" can be rewritten more explicitly as:
   a. "There does not exist a match in which the home team and away team are the same"

2. In predicate logic: $\nexists m \in M$, m.home_team=m.away_team, where M is the set of Matches

3. Which as a constraint can be written in relational algebra as:

$$\sigma_{\text{home_team=away_team}}(\text{Matches}) = \emptyset$$

m.home_team = m.away_team $\nexists m \in M$

Source: Garcia-Molina et al. (2007), §2.5.4

# Example 3: Constraints with Joins

**Constraint**: In a database of NHL hockey matches, no team called Nordiques can play at home after 1995
**Relations**: Matches(home_team, away_team, date), Team(id, city, name, year_founded)

Source: Garcia-Molina et al. (2007), §2.5.4

# Example 3: Constraints with Joins

**Constraint**: In a database of NHL hockey matches, no team called Nordiques can play at home after 1995
**Relations**: Matches(home_team, away_team, date), Team(id, city, name, year_founded)

1. "No team called Nordiques can play at home after 1995" can be rewritten more explicitly as:
   a. "There does not exist a match since 1995 in which either team is called the Nordiques"

This logical structure will lead to our constraint form: Query = ∅

Source: Garcia-Molina et al. (2007), §2.5.4

# Example 3: Constraints with Joins

**Constraint**: In a database of NHL hockey matches, no team called Nordiques can play at home after 1995
**Relations**: Matches(home_team, away_team, date), Team(id, city, name, year_founded)

1. "No team called Nordiques can play at home after 1995" can be rewritten more explicitly as:
   a. "There does not exist a match since 1995 in which either team is called the Nordiques"

This logical structure will lead to our constraint form: Query = ∅

Conditions involve attributes from different relations ⇒ need a join

Source: Garcia-Molina et al. (2007), §2.5.4

# Example 3: Constraints with Joins

**Constraint**: In a database of NHL hockey matches, no team called Nordiques can play at home after 1995
**Relations**: Matches(home_team, away_team, date), Team(id, city, name, year_founded)

1. "No team called Nordiques can play at home after 1995" can be rewritten more explicitly as:
   a. "There does not exist a match since 1995 in which either team is called the Nordiques"

$$\sigma_{\text{name} = \text{'Nordiques'} \wedge \text{date} > \text{'1995-12-31'}}(\text{Team} \bowtie_{\text{home_team} = \text{id} \vee \text{away_team} = \text{id}} \text{Match}) = \emptyset$$

Condition on name — Condition on date — Join relations — $\nexists m \in M$

Source: Garcia-Molina et al. (2007), §2.5.4

# Formally express this constraint

**Constraint**: In a database of NHL hockey matches, teams cannot play matches unless they have already been founded

**Relations**: Matches(home_team, away_team, date)
Team(id, city, name, year_founded)

Time Allotted: 120 seconds

# Example 4: Self-Joins

**Constraint**: In a database of NHL hockey matches, *derby matches** are not permitted
**Relations**: Matches(home_team, away_team, date), Team(id, city, name, year_founded)

**A derby match is one where two teams from the same city/region play against each other*

Source: Garcia-Molina et al. (2007), §2.5.4

# Example 4: Self-Joins

**Constraint**: In a database of NHL hockey matches, *derby matches** are not permitted
**Relations**: Matches(home_team, away_team, date), Team(id, city, name, year_founded)

1. "Derby matches are not permitted" can be rewritten more explicitly as:
   a. "There does not exist a match since 1995 in which both teams have the same city"

Comparing information from two tuples of the same relation instance requires joining the relation to itself: conditions have to apply to a single tuple

Need city from two distinct teams

**A derby match is one where two teams from the same city/region play against each other*

Source: Garcia-Molina et al. (2007), §2.5.4

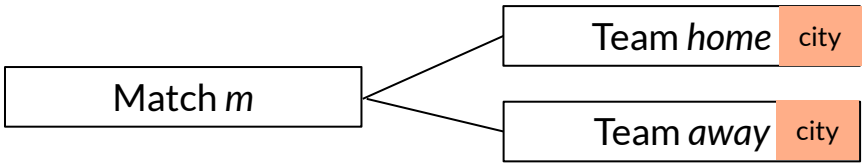
# Example 4: Self-Joins

**Constraint**: In a database of NHL hockey matches, *derby matches** are not permitted
**Relations**: Matches(home_team, away_team, date), Team(id, city, name, year_founded)

1. "Derby matches are not permitted" can be rewritten more explicitly as:
   a. "There does not exist a match since 1995 in which both teams have the same city"

Comparing information from two tuples of the same relation instance requires joining the relation to itself: conditions have to apply to a single tuple

Need city from two distinct teams




These should be equal

**A derby match is one where two teams from the same city/region play against each other*

Source: Garcia-Molina et al. (2007), §2.5.4

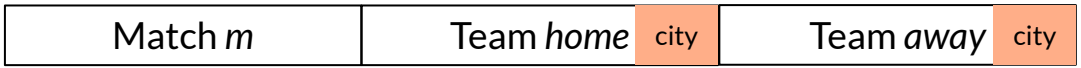
# Example 4: Self-Joins

**Constraint**: In a database of NHL hockey matches, *derby matches** are not permitted
**Relations**: Matches(home_team, away_team, date), Team(id, city, name, year_founded)

1. "Derby matches are not permitted" can be rewritten more explicitly as:
   a. "There does not exist a match since 1995 in which both teams have the same city"

Comparing information from two tuples of the same relation instance requires joining the relation to itself: conditions have to apply to a single tuple

Need city from two distinct teams




We can write conditions on tuples that concatenate ("join") a match with info about the home team and info about the away team

**A derby match is one where two teams from the same city/region play against each other*

Source: Garcia-Molina et al. (2007), §2.5.4

# Example 4: Self-Joins

**Constraint**: In a database of NHL hockey matches, *derby matches** are not permitted
**Relations**: Matches(home_team, away_team, date), Team(id, city, name, year_founded)

1. "Derby matches are not permitted" can be rewritten more explicitly as:
   a. "There does not exist a match since 1995 in which both teams have the same city"

$$\sigma_{\text{home.city = away.city}}(\rho_{\text{home}}(\text{Team}) \bowtie_{\text{home.id = home_team}} \text{Match} \bowtie_{\text{away.id = away_team}} \rho_{\text{away}}(\text{Team})) = \emptyset$$

| Match *m* | Team *home* | city | Team *away* | city |
|---|---|---|---|---|

We can write conditions on tuples that concatenate ("join") a match with info about the home team and info about the away team

**A derby match is one where two teams from the same city/region play against each other*

Source: Garcia-Molina et al. (2007), §2.5.4

# Expressing key & referential integrity constraints

# Example 5: Key/uniqueness constraint

**Constraint**: In a database of NHL hockey matches, every team name must be unique
**Relations**: Team(id, city, name, year_founded)

Source: Garcia-Molina et al. (2007), §2.5.3

# Example 5: Key/uniqueness constraint

**Constraint**: In a database of NHL hockey matches, every team name must be unique
**Relations**: Team(id, city, name, year_founded)

1. "Every team name must be unique" can be rewritten more explicitly as:
   a. "There does not exist a pair of different teams in which both teams have the same name"

Comparing information from two tuples of the same relation instance requires joining the relation to itself, but with care to ensure the two tuples are distinct

Need name from two distinct teams

Source: Garcia-Molina et al. (2007), §2.5.3

# Example 5: Key/uniqueness constraint

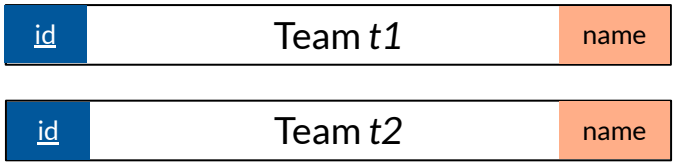
**Constraint**: In a database of NHL hockey matches, every team name must be unique
**Relations**: Team(id, city, name, year_founded)

1. "Every team name must be unique" can be rewritten more explicitly as:
   a. "There does not exist a pair of different teams in which both teams have the same name"

Comparing information from two tuples of the same relation instance requires joining the relation to itself, but with care to ensure the two tuples are distinct

Need name from two distinct teams

If these are different, then we have two distinct teams




These should be equal

Source: Garcia-Molina et al. (2007), §2.5.3

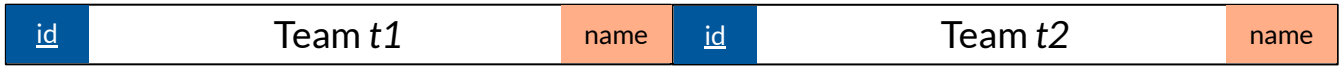
# Example 5: Key/uniqueness constraint

**Constraint**: In a database of NHL hockey matches, every team name must be unique
**Relations**: Team(id, city, name, year_founded)

1. "Every team name must be unique" can be rewritten more explicitly as:
   a. "There does not exist a pair of different teams in which both teams have the same name"

Comparing information from two tuples of the same relation instance requires joining the relation to itself, but with care to ensure the two tuples are distinct

Need name from two distinct teams




We can write conditions on tuples that concatenate ("join") pairs of distinct teams

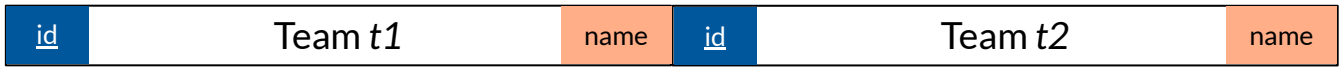
Source: Garcia-Molina et al. (2007), §2.5.3

# Example 5: Key/uniqueness constraint

**Constraint**: In a database of NHL hockey matches, every team name must be unique
**Relations**: Team(id, city, name, year_founded)

1. "Every team name must be unique" can be rewritten more explicitly as:
   a. "There does not exist a pair of different teams in which both teams have the same name"

$$\sigma_{t1.name = t2.name}(\rho_{t1}(Team) \bowtie_{t1.id > t2.id} \rho_{t2}(Team)) = \emptyset$$




We can write conditions on tuples that concatenate ("join") pairs of distinct teams

Source: Garcia-Molina et al. (2007), §2.5.3

# Example 6: Referential Integrity constraint

**Constraint**: In a database of NHL hockey matches, the home team of every match must exist in Teams
**Relations**: Matches(home_team, away_team, date), Team(id, city, name, year_founded)

Source: Garcia-Molina et al. (2007), §2.5.2

# Example 6: Referential Integrity constraint

**Constraint**: In a database of NHL hockey matches, the home team of every match must exist in Teams
**Relations**: Matches(home_team, away_team, date), Team(id, city, name, year_founded)

1. "The home team of every match must exist in Teams" can be rewritten more explicitly as:
   a. "There does not exist a match in which home_team is not an id in the Teams relation"

Source: Garcia-Molina et al. (2007), §2.5.2

# Example 6: Referential Integrity constraint

**Constraint**: In a database of NHL hockey matches, the home team of every match must exist in Teams
**Relations**: Matches(home_team, away_team, date), Team(id, city, name, year_founded)

1. "The home team of every match must exist in Teams" can be rewritten more explicitly as:
   a. "There does not exist a match in which home_team is not an id in the Teams relation"

This doesn't work! We cannot join tuples based on *not finding a match*!*

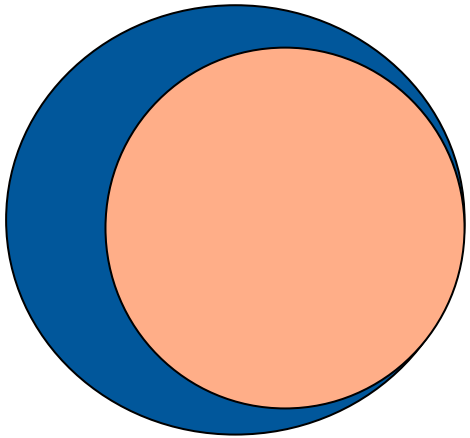
*An outer join could accomplish this, but we don't have that operator in relational algebra

Source: Garcia-Molina et al. (2007), §2.5.2

# Example 6: Referential Integrity constraint

**Constraint**: In a database of NHL hockey matches, the home team of every match must exist in Teams
**Relations**: Matches(home_team, away_team, date), Team(id, city, name, year_founded)

1. "The home team of every match must exist in Teams" can be rewritten more explicitly as:
   a. "There does not exist a match in which home_team is not an id in the Teams relation"



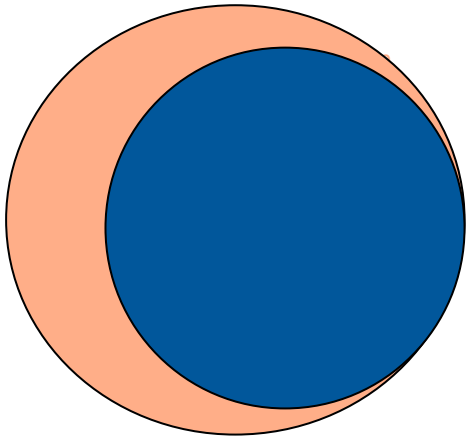
This constraint is easier to reason about in terms of the entire sets, rather than conditions on individual tuples

Source: Garcia-Molina et al. (2007), §2.5.2

# Example 6: Referential Integrity constraint

**Constraint**: In a database of NHL hockey matches, the home team of every match must exist in Teams
**Relations**: Matches(home_team, away_team, date), Team(id, city, name, year_founded)

1. "The home team of every match must exist in Teams" can be rewritten more explicitly as:
   a. "There does not exist a match in which home_team is not an id in the Teams relation"



$\pi_{home_team}(Matches) \setminus \pi_{id}(Team) = \emptyset$

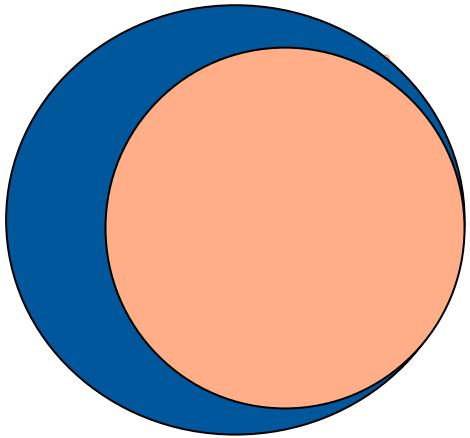
It should not be possible for this condition to arise between the two sets

Source: Garcia-Molina et al. (2007), §2.5.2

# Example 6: Referential Integrity constraint

**Constraint**: In a database of NHL hockey matches, the home team of every match must exist in Teams
**Relations**: Matches(home_team, away_team, date), Team(id, city, name, year_founded)

1. "The home team of every match must exist in Teams" can be rewritten more explicitly as:
   a. "There does not exist a match in which home_team is not an id in the Teams relation"



$\pi_{home_team}(Matches) \subseteq \pi_{id}(Team)$

Rearranging to the alternative form for expressing constraints is clearer in this case

Source: Garcia-Molina et al. (2007), §2.5.2

# Expressing functional dependencies

# Example 7: Functional Dependency

**Constraint**: In a database of NHL hockey matches, city name → year_founded
**Relations**: Team(id, city, name, year_founded)

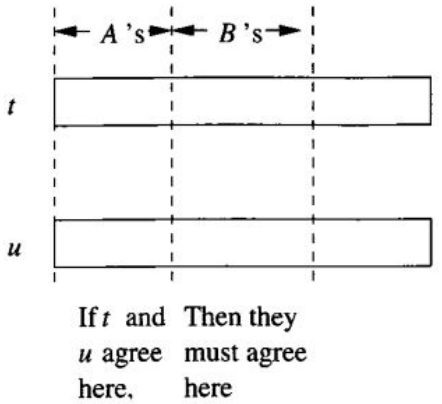
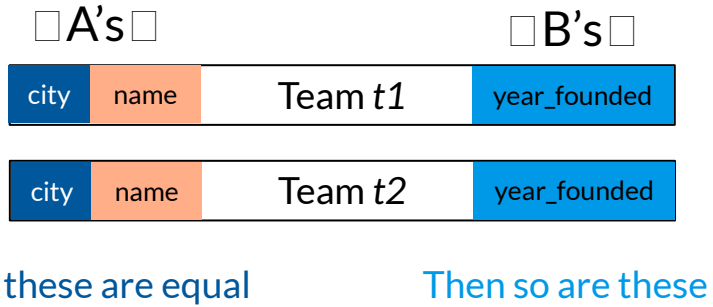



Figure 3.1: The effect of a functional dependency on two tuples.

Source: Garcia-Molina et al. (2007), §2.5.2

# Example 7: Functional Dependency

**Constraint**: In a database of NHL hockey matches, city name → year_founded
**Relations**: Team(id, city, name, year_founded)

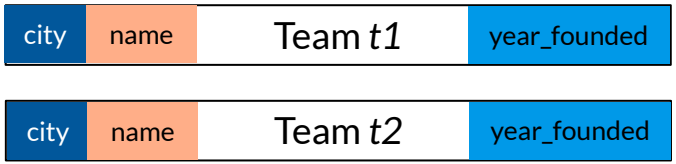



Source: Garcia-Molina et al. (2007), §2.5.2

# Example 7: Functional Dependency

**Constraint**: In a database of NHL hockey matches, city name → year_founded
**Relations**: Team(id, city, name, year_founded)

1. We need to assert that two conditions cannot simultaneously be true
   a. Two tuples match on city and name
   b. Those tuples differ on year_founded




Source: Garcia-Molina et al. (2007), §2.5.2

# Example 7: Functional Dependency

**Constraint**: In a database of NHL hockey matches, city name → year_founded
**Relations**: Team(id, city, name, year_founded)

1. We need to assert that two conditions cannot simultaneously be true
   a. Two tuples match on city and name
   b. Those tuples differ on year_founded




If these match t1

This cannot differ
From t1.year_founded

Source: Garcia-Molina et al. (2007), §2.5.2

# Example 7: Functional Dependency

**Constraint**: In a database of NHL hockey matches, city name → year_founded
**Relations**: Team(id, city, name, year_founded)

1. We need to assert that two conditions cannot simultaneously be true
   a. Two tuples match on city and name
   b. Those tuples differ on year_founded

$$\sigma_{t1.year_founded <> t2.year_founded}(\rho_{t1}(Team) \bowtie_{t1.city = t2.city \wedge t1.name = t2.name} \rho_{t2}(Team)) = \emptyset$$




If these match t1

This cannot differ
From t1.year_founded

Source: Garcia-Molina et al. (2007), §2.5.2

# Example 7: Functional Dependency

**Constraint**: In a database of NHL hockey matches, city name → year_founded
**Relations**: Team(id, city, name, year_founded)

1. We need to assert that two conditions cannot simultaneously be true
   a. Two tuples match on city and name
   b. Those tuples differ on year_founded

But differ on B's

If two tuples match on A's

Then A → B is violated

$$\sigma_{t1.year_founded <> t2.year_founded}(\rho_{t1}(Team) \bowtie_{t1.city = t2.city \wedge t1.name = t2.name} \rho_{t2}(Team)) = \emptyset$$




Source: Garcia-Molina et al. (2007), §2.5.2

# Summary / Exercise

# Formally express this constraint

**Constraint**: In a database of NHL hockey matches, express: *home_team → name*

**Relations**: Matches(home_team, away_team, date)
Team(id, city, name, year_founded)

Time Allotted: 180 seconds